

LISTS, STRINGS, AND DICTIONARIES Meta

COMPUTER SCIENCE MENTORS 61A

February 24, 2026 – February 27, 2026

Example Timeline

- Teaching tips:

Each problem's "Suggested Time" is not assigned with the 1 hour time for sections in mind. It is more of a note of how long each individual problem may take. That is, if you added all the "Suggested Times" in the coding section, it could very well be greater than 1 hour (especially in more complicated worksheets in the future).

However, the time in brackets will add up to 50 minutes, and is intended to be a suggested outline for your section.

Also, the worksheet feedback form is linked at the bottom of this worksheet!

- Intros, Ice Breakers, & Logistics [2 min]

Start off with an icebreaker of your choice!

- Mini-lecture: Lists [5 min]
- Q1: WWPDP Lists [5 min]
- Q2: List Comprehensions [7 min]
- Q3: List Recursion [7 min]
- Mini-Lecture: Strings [5 min]
- Q4: WWPDP Strings [5 min]
- Mini-Lecture: Dictionaries [5 min]
- Q5: WWPDP Dictionary [3 min]
- Q6: Dictionary Comprehension [If Extra Time]

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> a = [1, 2, 3]
>>> a
```

```
[1, 2, 3]
```

```
>>> a[2]
```

```
3
```

```
>>> a[:2]
```

```
[1, 2]
```

```
>>> len(a)
```

```
3
```

```
>>> c = [x * 2 for x in a]
>>> c
```

```
[1, 4, 6]
```

```
>>> sum(c)
>>> max(c)
>>> all(c)
>>> all([0, 1, 2, 3])
```

```
11
6
True
False
```

```
>>> b = [1, 2, 3, a, 4]
>>> b
```

```
[1, 2, 3, [1, 2, 3], 4]
```

```
>>> b[3][2]
```

```
3
```

Teaching Tips

1. The goal of this problem is to introduce lists, box and pointer diagrams, list indexing and slicing, list comprehensions, aggregation functions, and nested lists.
2. Encourage students to draw box and pointer diagrams if they seem stuck, but it is not necessary to draw a box and pointer diagram for all questions.

Suggested Time: 7 min

2. Write a list comprehension that accomplishes each of the following tasks.

- (a) Square all the elements of a given list, `lst`. First, write a for loop that accomplishes this task using the **range** function. Then, write a for loop using the **in** keyword. Finally, convert it into a list comprehension.

```
[x ** 2 for x in lst]
```

- (b) Compute the sum of all even numbers in the list `lst`. Is there a function you can apply to a list to get the sum of all elements?

```
sum([x for x in lst if x % 2 == 0])
```

- (c) Return a list of lists such that `a = [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]`.

```
a = [[x for x in range(y)] for y in range(1, 6)]
```

- (d) Return the same list as above, except now excluding every instance of the number 2: `b = [[0], [0, 1], [0, 1], [0, 1, 3], [0, 1, 3, 4]]`.

```
b = [[x for x in range(y) if x != 2] for y in range(1, 6)]
```

Teaching Tips

1. For the first subquestion, have students write a for loop both using the range function and also using the in keyword.
2. It may be helpful to start with the basic list comprehension template of `[<expr> for <item> in <iterable> if <condition>]`
3. The list of list questions are tricky, so try nudging your students in the right direction by reminding them that it is completely possible to nest list comprehensions.

Suggested Time: 7 min

3. Complete the definition for `all_true`, which takes in a list `lst` and returns `True` if there are no `False`-y values in the list and `False` otherwise. Make sure that your implementation is recursive. What is the name of the built-in Python function that does this?

```
def all_true(lst):  
    '''  
    >>> all_true([True, 1, 'True'])  
    True  
    >>> all_true([1, 0, 1])  
    False  
    >>> all_true([])  
    True  
    '''
```

```
    if not lst:  
        return True  
    elif not lst[0]:  
        return False  
    else:  
        return all_true(lst[1:])
```

The built-in Python function `all` does the same thing that `all_true` does!

Teaching Tips

1. It may be helpful to review recursion and discuss what the base case and recursive case should be.

Suggested Time: 7 min

1. What would Python display?

```
>>> a = "hello world!"  
>>> a[4]
```

'o'

```
>>> a = "hello world!"  
>>> a[6:11]
```

'world'

```
>>> len(a)
```

12

```
>>> 'lo' in a
```

True

Teaching Tips

1. The goal of this problem is to introduce dictionaries.
2. Encourage students to draw box and pointer diagrams if they seem stuck, but it is not necessary to draw a box and pointer diagram for all questions.

Suggested Time: 5 min

1. What would Python display?

```
>>> a = {1: "one", 2: "two", 3: "three"}
>>> a[2]
```

```
"two"
```

```
>>> b = {"five": 5, "six": [1,2,3], "seven": 7}
>>> b["five"]
```

```
5
```

```
>>> b["six"][2]
```

```
3
```

Teaching Tips

1. The goal of this problem is to introduce dictionaries.
2. Encourage students to draw box and pointer diagrams if they seem stuck, but it is not necessary to draw a box and pointer diagram for all questions.

Suggested Time: 3 min

2. Using a dictionary comprehension, complete the function below. It takes a single-argument function `f` and a dictionary `d`, and returns a new dictionary containing only the entries whose keys are even numbers, with `f` applied to their values.

```
def apply_to_even(f, d):
    '''
    >>> apply_to_even(lambda x: x**2, {1:3, 2:5, 4:2, 3:2})
    {2: 25, 4: 4}
    '''
    return {__ : ____ for ____ in ____ if _____}
```

```
def apply_to_even(f, d):
    return {i: f(d[i]) for i in d if i % 2 == 0}
```

Suggested Time: 5 min (If extra time)